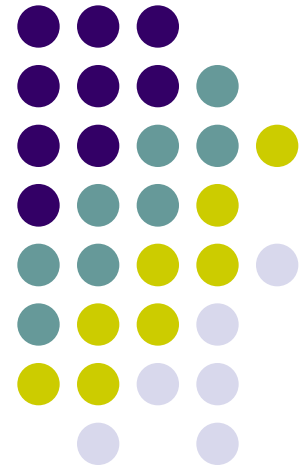
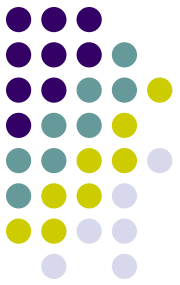


LẬP TRÌNH CĂN BẢN

Phần 2 - Chương 7 KIỂU CON TRỎ

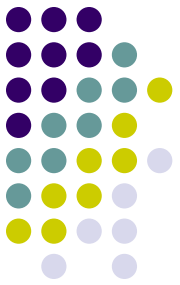




Nội dung chương này

- Giới thiệu kiểu dữ liệu con trỏ
- Khai báo và sử dụng biến con trỏ
- Con trỏ và mảng
- Con trỏ và tham số hình thức của hàm

Giới thiệu kiểu dữ liệu con trỏ (pointer) (1)

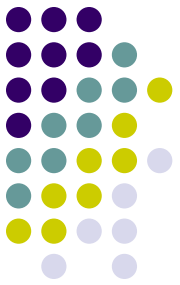


- 1 con trỏ là 1 biến được dùng để chứa địa chỉ của ô nhớ trong bộ nhớ.
- Kích thước của biến con trỏ luôn là 2 byte.
- Ví dụ 1:

```
float a=3.145;  
float* ptr;      //ptr is a pointer  
ptr=&a;
```



Giới thiệu kiểu dữ liệu con trỏ (2)



- Ví dụ 2:

```
main()
{
    int a = 5;
    int b = 6;
    int* c;

    // c points to a
    c = &a;
}
```

In Linux

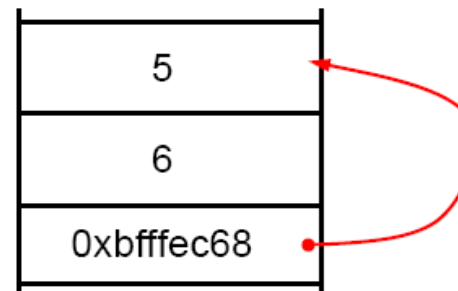
defines **c** as a pointer to a variable of type **int**

address operator retrieves memory location of variable **a**

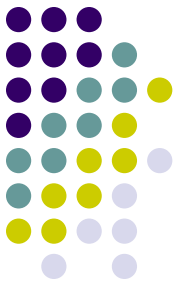
Variable

↓
a (0xbffec68)
b (0xbffec64)
c (0xbffec60)

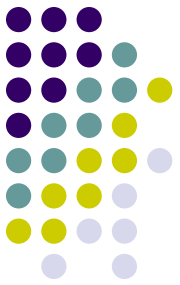
Memory



Khai báo và sử dụng biến con trỏ



- *Khai báo biến con trỏ*
- *Các thao tác trên con trỏ*



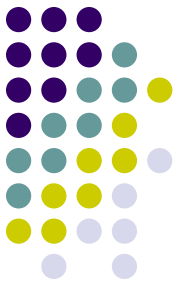
Khai báo biến con trỏ

```
int a, b, *pa, *pb; //pa và pb sẽ chỉ đến biến int  
float f, *pf;      //pa và pb sẽ chỉ đến biến float  
void *ptr;         //ptr sẽ chỉ vào bất kỳ biến kiểu gì
```

- Cú pháp:

<Kiểu> *<Tên con trỏ>;

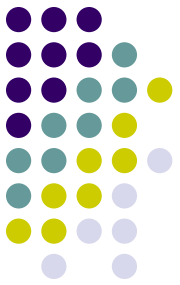
- **Ý nghĩa:** Khai báo một biến có tên là *Tên con trỏ* dùng để chứa địa chỉ của các biến có kiểu *Kiểu*.



Các thao tác trên con trỏ

- *Gán địa chỉ của biến cho biến con trỏ*
- *Nội dung của ô nhớ con trỏ chỉ tới*
- *Cấp phát vùng nhớ cho biến con trỏ*
- *Cấp phát lại vùng nhớ cho biến con trỏ*
- *Giải phóng vùng nhớ cho biến con trỏ*
- *Một số phép toán trên con trỏ*

Gán địa chỉ của biến cho biến con trỏ + Nội dung của ô nhớ con trỏ chỉ tới (1)



- Dùng & để lấy ra địa chỉ bộ nhớ (memory address) của 1 biến
`int a=6;`
`int* c= &a; // &a là địa chỉ bộ nhớ của biến a`
- Dùng * để truy cập (access) đến nội dung (content) của biến mà 1 con trỏ đang chỉ đến

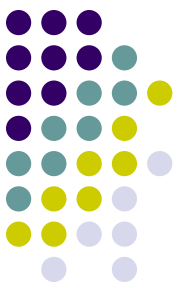
`int a=6;`

`int* c= &a;`

`*c=7; /*Thay đổi nội dung của biến a bằng cách
dùng địa chỉ của nó được chứa trong con trỏ c*/`

tương đương với

`a=7;`



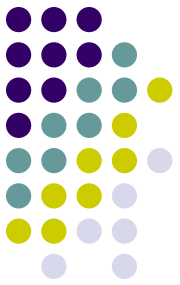
Gán địa chỉ của biến cho biến con trỏ + Nội dung của ô nhớ con trỏ chỉ tới (2)

```
#include <stdio.h>
#include <conio.h>
int main(){
    int a,b,*pa,*pb;
    a=2;
    b=3;
    clrscr();
    printf("\nGia tri cua bien a=%d \nGia tri cua bien b=%d ",a,b);
    pa=&a;
    pb=&b;
    printf("\nNoi dung cua o nho con tro pa tro toi=%d",*pa);
    printf("\nNoi dung cua o nho con tro pb tro toi=%d ",*pb);
    *pa=20; // Thay doi gia tri cua *pa
    *pb=20; // Thay doi gia tri cua *pb
    printf("\nGia tri moi cua bien a=%d \n
           Gia tri moi cua bien b=%d ",a,b); // a, b thay doi theo
    getch();
    return 0;
}
```

C:\ TC.EXE

```
Gia tri cua bien a=2
Gia tri cua bien b=3
Noi dung cua o nho con tro pa tro toi=2
Noi dung cua o nho con tro pb tro toi=3
Gia tri moi cua bien a=20
Gia tri moi cua bien b=20
```

Gán địa chỉ của biến cho biến con trỏ + Nội dung của ô nhớ con trỏ chỉ tới (3)



- **Lưu ý:**

```
int    Bien_Nguyen;  
float  *Con_Tro_Thuc;  
...  
Con_Tro_Thuc=&Bien_Nguyen;
```

=> Error!

Vì đã cho 1 con trỏ chỉ đến 1 biến khác kiểu với nó

Cấp phát vùng nhớ cho biến con trỏ(1)



- **Có 2 cách để dùng được biến con trỏ**

1. Cho nó chứa địa chỉ của 1 vùng nhớ đang tồn tại

```
int a=6;
```

```
int* c= &a; // &a là địa chỉ bộ nhớ của biến a
```

2. Cấp phát 1 vùng nhớ mới, rồi cho con trỏ chỉ đến

```
int* ptr;
```

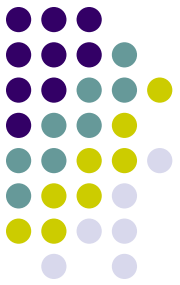
```
ptr = (int*)malloc(sizeof(int));
```

```
*ptr=6;
```

pointer **ptr**



FFFA (address)



Cấp phát vùng nhớ cho biến con trỏ(2)

- **void *malloc(size_t size):** Cấp phát vùng nhớ có kích thước là size (byte)
- **void *calloc(size_t nitems, size_t size):** Cấp phát vùng nhớ có kích thước là nitems*size (byte)

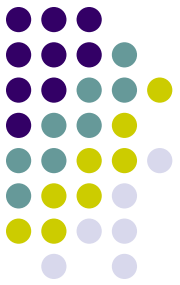
- **Ví dụ:**

```
int a, *pa, *pb;
```

```
pa = (int*)malloc(sizeof(int)); /* Cấp phát vùng nhớ có kích thước  
bằng với kích thước của một số nguyên */
```

```
pb= (int*)calloc(10, sizeof(int)); /* Cấp phát vùng nhớ có thể chứa  
được 10 số nguyên*/
```

Cấp phát lại vùng nhớ cho biến con trỏ

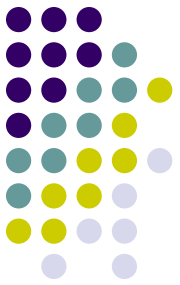


```
int a, *pa;  
pa = (int*)malloc(sizeof(int)); /*Cấp phát vùng nhớ có  
                                kích thước 2 byte*/  
pa = realloc(pa, 6);           /*Cấp phát lại vùng nhớ có  
                                kích thước mới là 6 byte*/
```

- **void *realloc(void *block, size_t size):**

- *Ý nghĩa:*

- Cấp phát lại 1 vùng nhớ do con trỏ *block* quản lý, vùng nhớ này có kích thước mới là *size*; khi cấp phát lại thì nội dung vùng nhớ trước đó được copy đến vùng nhớ mới.
- Kết quả trả về của hàm là địa chỉ đầu tiên của vùng nhớ mới. Địa chỉ này có thể khác với địa chỉ được chỉ ra khi cấp phát ban đầu. Kết quả là NULL nếu không cấp phát được.



Giải phóng vùng nhớ cho biến con trỏ

- **void free(void *block):** Giải phóng vùng nhớ được quản lý bởi con trỏ block

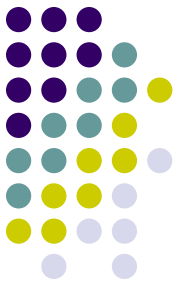
- **Ví dụ**

free(pa);

free(pb);

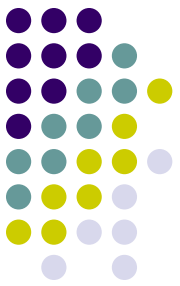
=> giải phóng vùng nhớ do

2 biến con trỏ pa & pb đang chỉ đến



Một số phép toán trên con trỏ

- Phép gán =
- Phép so sánh == và !=
- Cộng, trừ con trỏ với 1 số nguyên
- Gán **NULL** cho 1 con trỏ



Phép gán và phép so sánh

- **Ví dụ:** Hiện tại ta có:

```
int a=10, b=15;
```

```
int *p, *q;
```

```
float *f;
```

```
p=&a; q=&b;
```

Bây giờ thì phép so sánh:

```
(p!=q) => true (1)
```

Thực hiện tiếp lệnh gán:

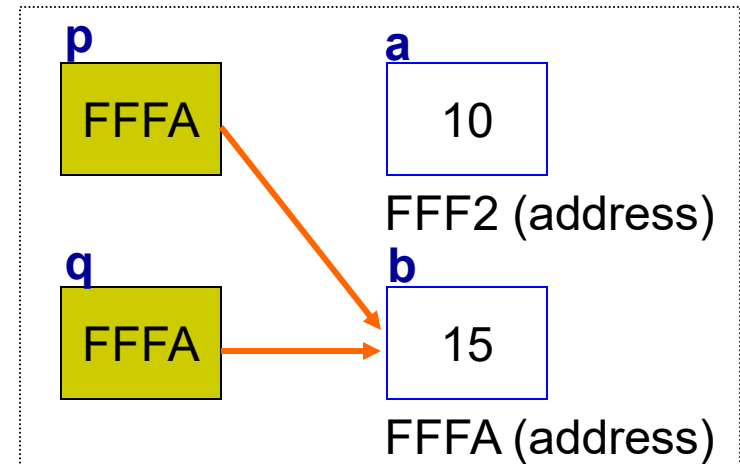
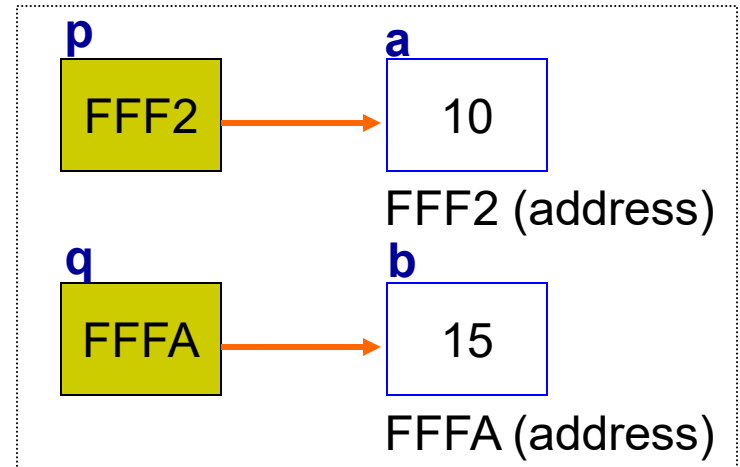
```
p=q;
```

Bây giờ thì:

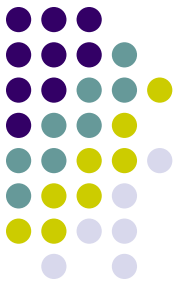
```
(p==q) => true (1)
```

Lệnh `f=p;` => **Error**, do khác kiểu

Nhưng lệnh `f=(float*)p;` => **No error**



Cộng, trừ con trỏ với 1 số nguyên



- Ta có thể cộng (+), trừ (-) 1 con trỏ với 1 số nguyên N nào đó
- Kết quả trả về là 1 con trỏ. Con trỏ này chỉ đến vùng nhớ cách vùng nhớ của con trỏ hiện tại N phần tử.
- **Ví dụ:** Cho đoạn chương trình sau:

```
int *pa;
```

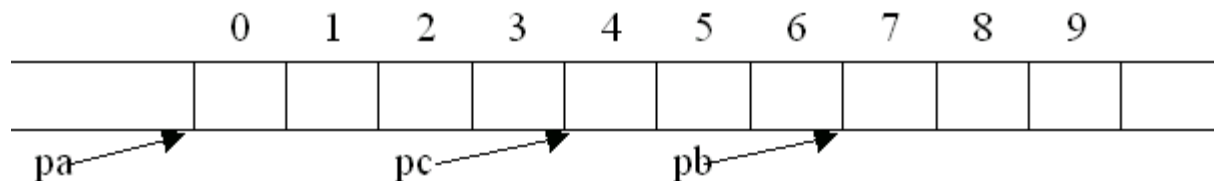
```
int *pb, *pc;
```

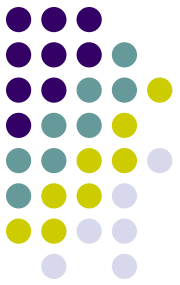
```
pa = (int*) malloc(20); /*Cấp phát vùng nhớ 20 byte=10 số nguyên*/
```

```
pb = pa + 7;
```

```
pc = pb - 3;
```

- Lúc này hình ảnh của pa, pb, pc như sau:



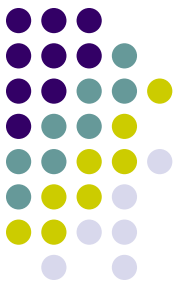


Gán NULL cho 1 con trỏ

- Ví dụ:

```
int x=25;  
int *ptr;  
ptr=&x;  
ptr=NULL;
```

- Lệnh gán **ptr=NULL** => cho con trỏ ptr không trỏ vào (không chứa địa chỉ) vùng nhớ nào cả



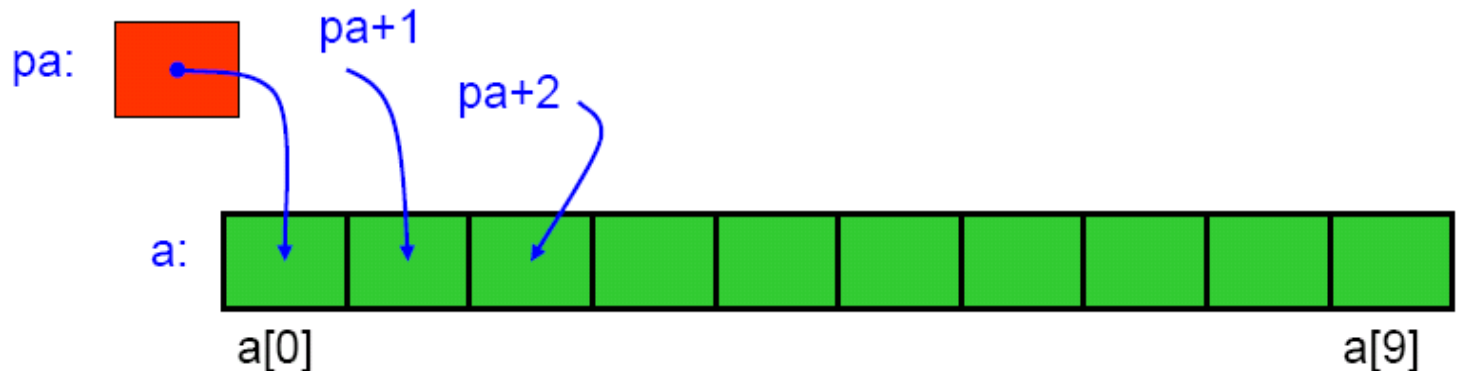
Con trỏ và mảng (1)

- Mảng và con trỏ có mối liên hệ với nhau
- Thay vì truy cập 1 phần tử mảng bằng chỉ số của nó, ta có thể dùng 1 con trỏ

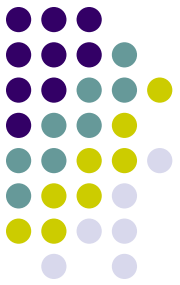
```
int a[10];  
int* pa;  
pa = &a[0];
```

Truy cập phần tử đầu tiên của mảng:

```
x = *pa; // Tương đương với x=a[0]
```



- Để truy cập phần tử thứ i , $a[i]$:
 $y = *(pa+i);$
- Chú ý: $pa+1$ không phải cộng 1 vào pa . Thay vào đó, địa chỉ là $pa+sizeof(*pa)$



Con trỏ và mảng (2)

- 1 mảng có thể được xem như một con trỏ. Con trỏ này đang chỉ đến phần tử đầu tiên của mảng. Do đó:

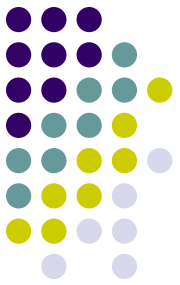
<code>pa = &a[0];</code>	$\Leftrightarrow$	<code>pa = a;</code>
<code>&a[i]</code>	$\Leftrightarrow$	<code>a+i</code>
<code>pa[i]</code>	$\Leftrightarrow$	<code>*(pa+i)</code>
<code>char s[]</code>	$\Leftrightarrow$	<code>char* s</code>

- 1 mảng có thể được dùng làm tham số cho 1 hàm

```
int a[10];
```

```
f(a);      // hoặc tương đương  
f(&a[0]);
```

```
void f(int* array)  
{  
    ...  
}  
// hoặc tương đương  
void f(int array[])  
{  
    ...  
}
```



Con trỏ và mảng (3)

- x sẽ có giá trị của a[i] nào, tại mỗi lệnh gán sau?

```
int a[10];  
int* pa = a;
```

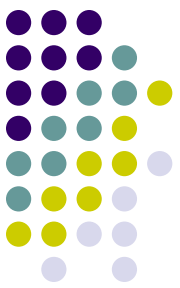
```
x = *pa;          // x = a[0]
```

```
pa++;  
x = *pa;          // x = a[1]
```

```
pa = pa+5;  
x = *pa;
```

```
pa = pa-4;  
x = *pa;
```

```
pa += 3;  
x = *pa;
```



Con trỏ và tham số hình thức của hàm (1)

- **Ví dụ 1:** Viết 1 hàm để tăng giá trị của biến lên 1

```
#include <conio.h>
#include <stdio.h>

void tangtri(int a, int *b){
    a=a+1;
    *b=*b+1;
    printf("Ben trong ham tangtri, tham so hinh thuc: a=%d b=%d \n",a,*b);
}

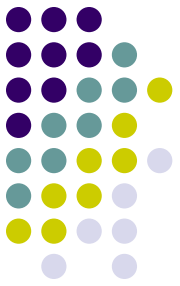
void main() {
    int x=5,y=5;
    clrscr();
    tangtri(x, &y);
    printf("Sau loi gọi ham trong main, tham so thuc te: x=%d y=%d",x,y);
    getch();
}
```



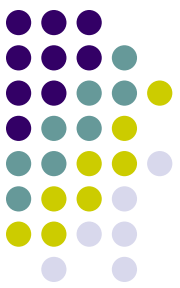
Turbo C++ IDE

```
Ben trong ham tangtri, tham so hinh thuc: a=6 b=6
Sau loi gọi ham trong main, tham so thuc te: x=5 y=6
```

Con trỏ và tham số hình thức của hàm (2)



- Đây là sự khác nhau giữa tham số a và b?



Con trỏ và tham số hình thức của hàm (3)

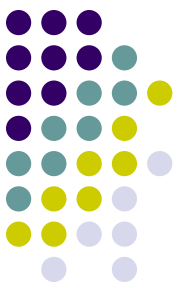
- **Ví dụ 2:** Viết hàm hoán đổi nội dung 2 biến

```
#include<stdio.h>
#include<conio.h>
void HoanVi(int a, int b){
    int c=a;
    a=b;
    b=c;
}

int main(){
    int m=20,n=30;
    clrscr();
    printf("Truoc khi goi ham m= %d, n= %d\n",m,n);
    HoanVi(m,n);
    printf("Sau khi goi ham m= %d, n= %d",m,n);
    getch();
    return 0;
}
```

 Turbo C++ IDE

Truoc khi goi ham m= 20, n= 30
Sau khi goi ham m= 20, n= 30_



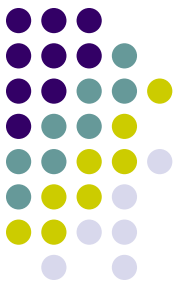
Con trỏ và tham số hình thức của hàm (4)

- **Ví dụ 3:** Viết lại hàm hoán vị như sau:

```
#include<stdio.h>
#include<conio.h>
void HoanVi(int *a, int *b){
    int c=*a;
    *a=*b;
    *b=c;
}

int main(){
    int m=20,n=30;
    clrscr();
    printf("Truoc khi goi ham m= %d, n= %d\n",m,n);
    HoanVi(&m,&n);
    printf("Sau khi goi ham m= %d, n= %d",m,n);
    getch();
    return 0;
}
```

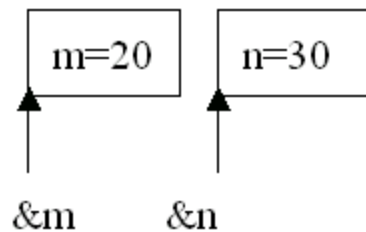
```
TC.EXE
Truoc khi goi ham m= 20, n= 30
Sau khi goi ham m= 30, n= 20
```



Con trỏ và tham số hình thức của hàm (5)

- Tại sao hàm trên có thể hoán vị được?

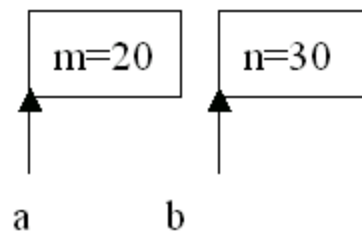
Trước khi gọi hàm



Khi gọi hàm

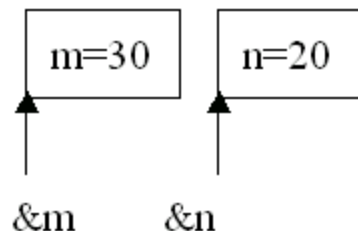
a=&m; b= &n;

Lúc này : *a=m; *b=n;



Đổi chỗ ta được :

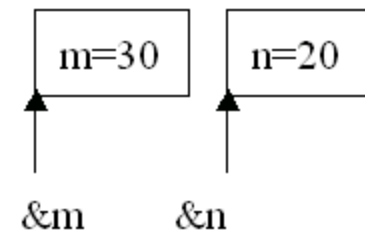
*a=m=30; *b=n=20;



Sau khi gọi hàm

Con trỏ a, b bị giải phóng

m, n đã thay đổi:



Hết chương

